

CYT-VX8 USER GUIDE

CYT-VX8

simplified virtual processor for education purpose



Version 1.0

Date of publication : May 2026

Written by Eva ANSERMIN & Romuald GRIGNON

Table of contents

Overview

◆ Purpose	6
◆ CYT-VX8 vs a real CPU	6
◆ Naming conventions	8
◆ Main diagram	10

Internal CPU architecture

Data Path **14**

◆ Arithmetic & Logic Unit	14
◆ Registers	15
◆ Data Path	15

Control Unit **17**

◆ Fetch	17
◆ Decode	17
◆ Execute	18

Addressing Unit **19**

◆ Code address register	19
◆ Load and Store instructions	20
◆ Using constants	21
◆ Jump instructions	20

Memory interface **22**

◆ Description	22
◆ Interface diagram	22

Instruction set

Description	26
Arithmetic - 2 registers	27
◆ ADD	27
◆ SUB	27
◆ MAX	28
◆ MIN	28
Arithmetic - 1 register	29
◆ INC	29
◆ DEC	29
◆ ABS	30
◆ NEG	30
Logic - 2 registers	31
◆ AND	31
◆ OR	31
◆ XOR	32
◆ NAND	32
◆ NOR	32
◆ XNOR	33
Logic - 1 register	34
◆ NOT	34
◆ LSL	34
◆ LSR	35
◆ ASR	35
◆ ROL	36
◆ ROR	36
Data comparisons	37
◆ CMP_LT	37
◆ CMP_LE	37
◆ CMP_EQ	38
◆ CMP_GE	38
◆ CMP_GT	38
◆ CMP_NE	39

Jump instructions	20
♦ JMP	40
♦ JEQ	40
♦ JNE	41
Memory access	42
♦ LOAD	42
♦ STOR	42
♦ PAGE	42
Others	43
♦ CNST	43
♦ NOP	43

Appendices

Instruction index	46
Figure Index	47
Assembly code examples	48
♦ Add & Subtract	48
♦ Change sign & Maximum value	48
♦ Bitwise Operations	49
♦ Sum of integers	49
♦ Switch case function	50

Overview

Purpose

This document purpose is to help students learn how a real CPU works. Obviously, this CPU is intended to be simplified, but not too much, in order to address all the different topics linked to CPU design in the course duration.

CYT-VX8 is a virtual CPU (it is not intended to be built for real). Nevertheless, a Python script has been created to compile bytecode from assembly code. This allows students to load bytecode in their simulations and run the code *for real*. All mnemonics and bytecode values are given in order to both build the circuit, and be able to generate byte code from assembly.

CYT-VX8 vs a real CPU

Our CPU is based on the Harvard architecture in order to split data and code memory access. This is a huge simplification in the whole circuit as we are able to get the next code instruction from memory + perform a read/write operation on data at the same time.

Nowadays real classic CPUs are based on Von Neumann architecture, where data and code are stored in the same memory, implying each data access has to stop code execution in order to be able to read/write. Von Neumann machines are slower but they save a lot of wires because there is only one memory bus. Moreover, as there is only one memory, it is easier to split memory between code and data. Briefly, hardware is more versatile to a lot of usages.

Moreover, the data memory device we want our CPU to be connected with, is expected to provide two data buses : one for reading from, and another for writing to. This additional design choice makes the whole circuit easier to implement. There is no need to handle multiple entities sharing the same bus.

In real life, for cost savings, a circuit allows to read and write on the same bus, needing a mechanism to handle which part of the circuit has the priority on the bus. Some patterns using multiplexers, or buffers with tri-state logic can be found.

Our instruction format is a fixed-size one. All instructions are made of 3 bytes (24bits). This is a real huge simplification too, because the fetch part has nothing to worry about instruction size. A lot of assembly languages (based on real CPUs or course) have variable-size instructions.

All registers are 8-bit length and can only contain integer values. Natural binary and two's complement representations are both possibles. All the processor instructions will last only 1-cycle to simplify the global behaviour. That means there is no "stall" signals coming and going across the whole circuit. Every clock cycle, a different instruction is executed. Though, operations such as multiplication or division are not available.

This simplification has one drawback : we cannot compute anything that needs more than one byte at a time. If we would develop a C language compiler for our CPU, any more-than-one byte variable would take a lot of operations (e.g. an arithmetic operation would take several cycles to be executed + we would need to manually handle the carry).

There is a specific register called RCMP that is used to store a comparison result. This register could be related to what we call flags in a real CPU : this register, even if this is an 8-bit register as R1-R7, only one bit is used here. This bit is used when the CPU executes jump instructions, in order to know if we have to branch or not. In some way : RCMP could illustrate the ZERO flag real CPUs use.

This flag topic is related to the previous paragraph : our CPU does not have any carry flag (a 1-bit register that could contain an overflow information about the previous operation) : this flag would be useful to handle more-than-one byte operations. This is a colossal limitation here. We could expect some great students to add such a flag mechanism ^_^.

In addition, to simplify all the addressing modes, immediate values for computation are not available : constants shall be loaded into a register before being used. Such a mechanism slows down the code execution because 1-cycle is needed to load the constant before using it with another register.

Another default of our CPU is the fact that it is impossible to read from/write to memory without a constant address given in the instruction. That means the data memory flows must be performed with static addresses : it is not yet possible to access the data memory using a register value (e.g. as a pointer). Our CPU is intended to understand real CPU behaviours, but is actually very far from being really useful.

A specific *code* limitation is the lack of indirect memory addressing mode : using a register to store the address of the source/destination (a.k.a. pointer in C language).

Our LOAD and STOR instructions only take a constant value in the instructions themselves. It is though impossible to get a data value from the memory using a register as source address, then increase this address and get the next value. In these conditions, it is impossible to loop through an array with our hardware. What can a developer do without pointers ;-) ?

The last chapter where our circuit seems very limited compared to the state-of-the-art is the fully addressable memory size. The current PC register is only expected to have a 8-bit size. That means, the maximum code size would be 24bits (instruction size) x 256 (PC size) = 768 bytes. With such a tiny RAM size, a lot of engineers will have to work to make the code fit into it ^_^. Anyway for the course needs, it is sufficient.

The data memory size is also accessed by a 8-bit address bus. That would limit the RAM size to 256-bytes. A **PAGE** instruction has been designed for a future use : some more registers could be added to the circuit, then the **PAGE** instruction would load one by one the complete page number (e.g. 3 bytes for the page) then **LOAD** and **STOR** instructions could access the lowest byte range, increasing the total RAM size (to 4Gbytes with this example). Such a design would slow the code execution everytime we want to access another page but could be useful for students to understand memory access limitations and performance compromises.

Naming conventions

In the following pages, some keywords are used to identify parts of the device, or parts of the instructions. Please refer to the current section if you need more information on part names. This list is sorted in alphabetical order.

ADDR8

This is an 8-bit value for an address to be given from either **LOAD**, **STOR** or **JNE**, **JEQ** instructions.

In the case of **LOAD** or **STOR** instructions, this address value will be the source (respectively the destination) of the data memory. **LOAD** will copy a byte from data memory to any register, and **STOR** will copy a register value into the data memory.

When using **JNE** or **JEQ** instructions, this address value will be the address of the next instruction (code memory) to be fetched.

Addressing Unit

This gathers all the circuits that make the CPU able to redirect inputs and outputs. It manages the data flow between all data important parts (memory, constants from instructions, registers). It is also in charge of fetching the next code instruction in order to provide it to the Control Unit. The PC register could (would? should?) be part of the data path, but it is in the addressing unit because PC is the link between all the CPU parts and it came to us that it would be easier to separate blocks like this for students.

CNST8

This is an 8-bit value for a constant to be stored in any register. This value is only handled when using the **CNST** instruction.

Control Unit

This gathers all the circuits that make the CPU able to fetch the next instruction, decode it and execute it. It combines sequential circuits (registers) and combinatory circuits to create the execute signals sent to the other parts (addressing unit, or data path).

Data Path

This gathers all the circuits that make the CPU able to select one or two inputs, process an operation with these inputs, and provides the output to the next part of the circuit. The ALU is at the heart of the data path as it performs all the available operations.

ID

Register storing the last decoded instruction in order to be executed during the next cycle. It is part of the Control unit.

IF

Register storing the last fetched instruction in order to be decoded during the next cycle. It is part of the Control unit.

PC

Register storing the program address (Program Counter). This is the address of the last fetched instruction (instruction currently loaded into the IF register). Due to our CPU design, the PC register has an 8-bit size.

R1, R2, ..., R7

These are the CPU internal register names. Each register has a specific number, respectively from 0 to 6. These registers can contain a 8-bit value each.

RCMP

This is the CPU internal register name for all the comparison results. This register could contain a 8-bit value but our simplified design only needs one (either the previous comparison was correct or not). This register number is hard-coded and equal to 7. It is never expected by the user to explicitly select this register as an output or input. This will be handled by the instruction bytecodes themselves.

#Rin1

This is the **selected register number** for the **first ALU input**. This register can be any from R1 to R7. As this value is intended to be used on 4 bits, the possible values are in the binary range **[0000, 0110]**, respectively.

#Rin2

This is the **selected register number** for the **second ALU input**. This register can be any from R1 to R7. As this value is intended to be used on 4 bits, the possible values are in the binary range **[0000, 0110]**, respectively.

#Rout

This is the **selected register number** to be modified. This register will contain any 8-bit value coming from either the ALU operation output, or a data memory byte (**LOAD** instruction) or any constant (**CNST** instruction). This register can be any from R1 to R7. As this value is intended to be used on 4 bits, the possible values are in the binary range **[0000, 0110]**, respectively.

Main diagram

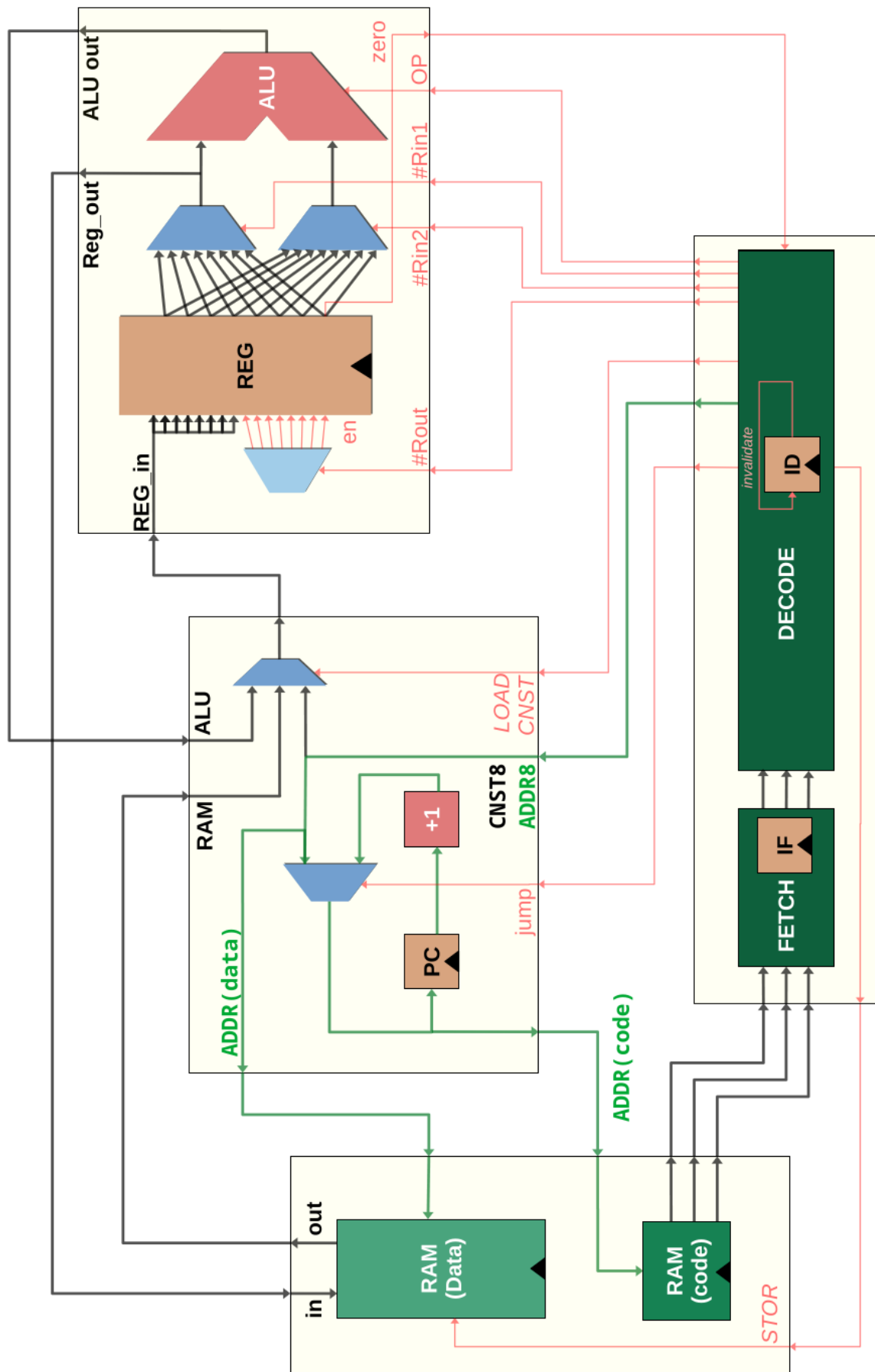
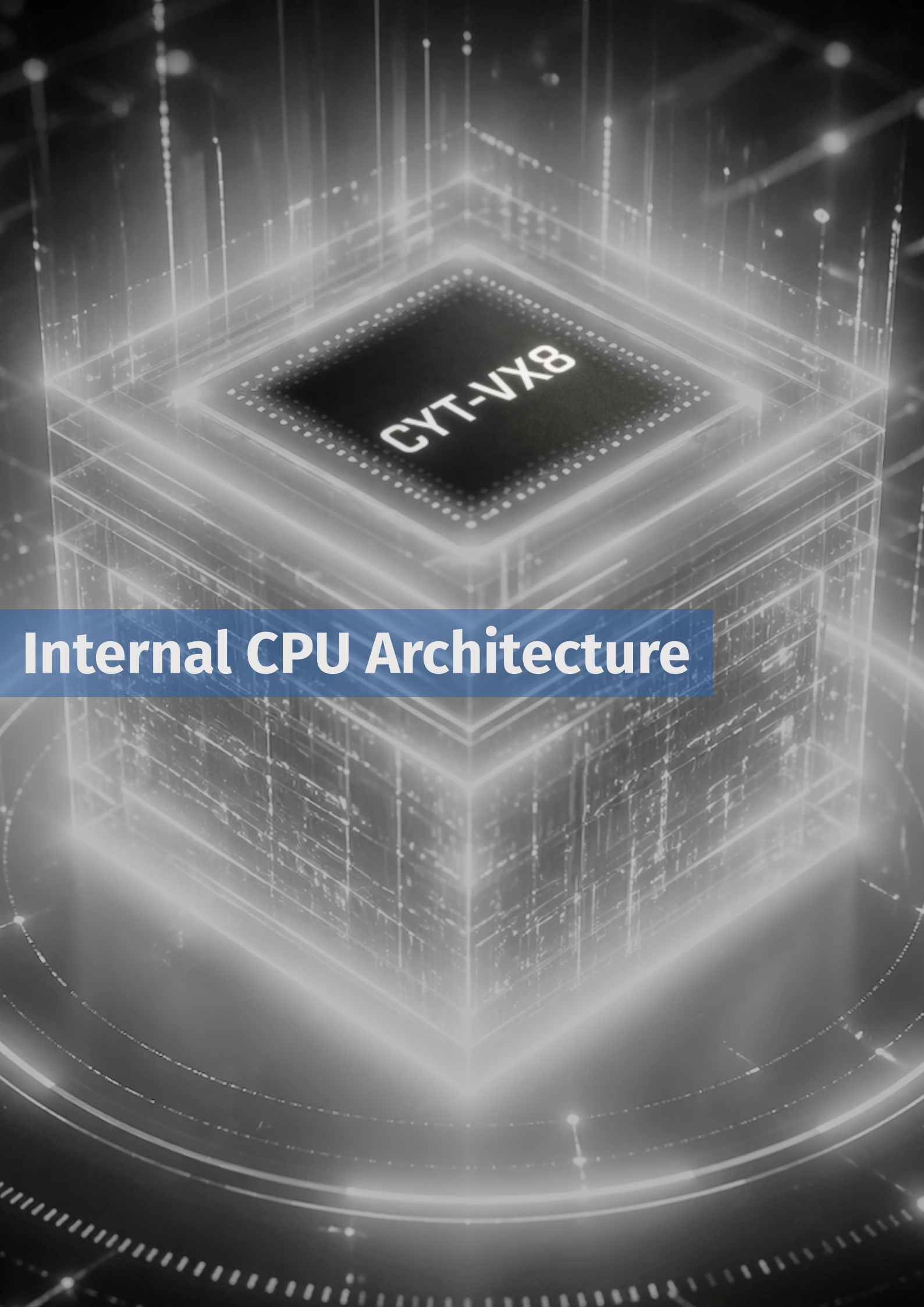


Figure : CYT-VX8 Full circuit

In the previous figure, we can see 4 main blocks that are part of the CYT-VX8 design :

- The data-path in the upper-right corner, in charge of all the data operations and register storage.
- The Control Unit in the bottom part, handling the instructions coming from the program, decoding them in order to drive all the other blocks.
- The memory block, on the left, used to store both data and code. This block is not intended to be inside the CPU itself, but linked to it.
- The Control Unit, that is in-between, redirecting data from/to the memory, the registers, or the instruction content itself.



CYT-VX8

Internal CPU Architecture

Data Path

In a CPU the "*data path*" contains all the logic blocks involved in the data computation and transport. These blocks have the responsibility to perform any instruction. In order to be able to execute the correct operation with the correct inputs and outputs, some control signals have to be available. Here follow the main functional blocks of the data path :

Arithmetic & Logic Unit

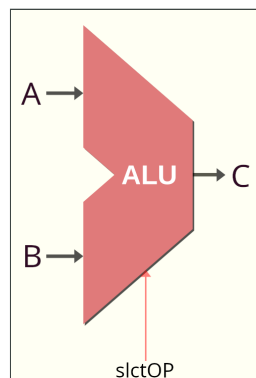


Figure : Arithmetic & Logic Unit (A.L.U.)

This unit contains all the data operations to perform such as addition, subtraction, logical shifts, comparisons, This is a pure combinatory circuit processing all the operations at the same time in parallel. There are two inputs (A & B) getting the values to be computed. The selected operation output will be redirected to the only one output (C), using a multiplexer (controlled by the slctOP signals in the figure). All the input and output sizes are 8-bits.

The slctOP signal shall have a size large enough to control all the defined ALU operations. The OP field of any instruction covers 6-bits, but the total number of ALU operations is 26, so 5 bits would be sufficient.

Registers

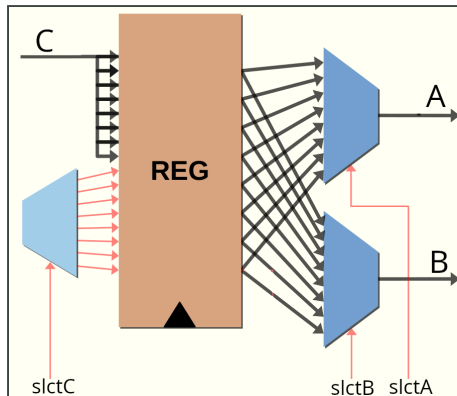


Figure : Data registers + multiplexers + decoder

The register block will contain the 8-bits registers. There shall be 7 registers named **R1** to **R7**, each one having an id number, respectively from 0 to 6. These registers will rely on latches to keep their values alive while the circuit is power supplied. Obviously this part of the circuit has a sequential behaviour based on the main clock cycle.

A specific register has to be created here : **RCMP**, that is the comparison result register. Technically, it will simplify the layout if this register has 8-bits, but in our design, this register only need 1-bit to keep the last comparison result (True or False). This register has the id number 7.

The main objective of this block is to make these registers to be selected (either to be read, or to be written, or both during the same clock cycle). This is achieved by the use of 2 multiplexeurs for the outputs (A & B) and one decoder for the input selection (C). The decoder shall be able to select no register at all, in case of cycles where no registers are needed to be written.

The **RCMP** register can be selected too, so it allows the whole circuit to choose between 8 registers of each 8-bits.

Another register could be here : the **PC** register. This register contains the current code instruction address. this register is very specific because it is in the middle of the whole behaviour : data-path /control unit / addressing block. So we have decided to keep it in the addressing block.

Data Path

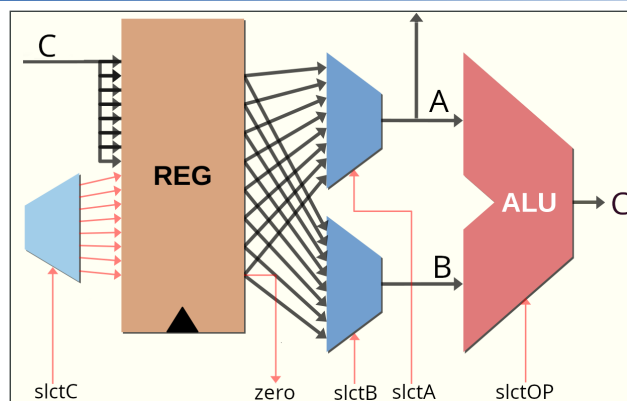


Figure : Full Data-path

The full data path block as we call it in this document, is the merge of the two previous blocks : Registers + ALU. This paragraph may seem useless, but if we focus on the rest of the CPU circuit we need to provide several features such as *register-to-memory* data copy, and jump operations.

The copy from register to memory needs the data path block to output a value. if you closely look at the register block, it is directly sent into the ALU inputs, and this last one only outputs a value as an operation result. So there is two choices here : either make the ALU being able to bypass an input value (designing a **PASS** operation that just does nothing), or make the data poath offering the ability to directly output a register value.

In our CPU design we have chosen the second one : in the previous figure you can see the A signal to be output from the data-path block for future copy operation.

In a real CPU, the jump operations are commonly duplicated, based on several flag values (last operation states such as ZERO, CARRY, UNDERFLOW, ...). In our CPU, we have two jump operations : **JNE** (jump if not equal) and **JEQ** (jump if equal), respectively branching the code if the last comparison result was not equal to zero (comparison was correct) or was equal to zero (comparison was incorrect). For specific course purpose, the last comparison result is stored in the **RCMP** register, and this result shall be accessible to the Control Unit : the *zero* flag shall be output from the data path block for future jump operation.

Control Unit

In a CPU, the control unit contains all the blocks needed to orchestrate the different instruction states. First the next instruction needs to be fetched from the code memory, then this instruction needs to be decoded in order to know if this is a valid one or not, and then finally it needs to be executed.

These three steps are a very huge simplification of what a real control unit can do, but it is sufficient to address some of the main topics about the control unit.

Fetch

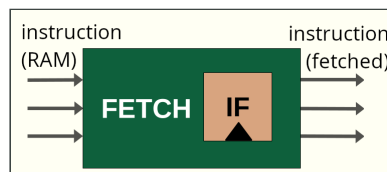


Figure : Fetch module

The *fetch* block is there to retrieve the correct number of bytes according to the instruction code. In a real CPU, this block reads exactly the number of bytes needed by the next instruction, due to the fact that instructions can have different sizes.

In our CPU, the instruction size is fixed and always 24-bits. So the fetch part is just a register storing the next 3 bytes from code memory. Each clock cycle, the **IF** register stores the next program instruction. On the next clock cycle, this instruction is provided to the next block : *decode*. In our case, this block simply reduces to a pure register.

Decode

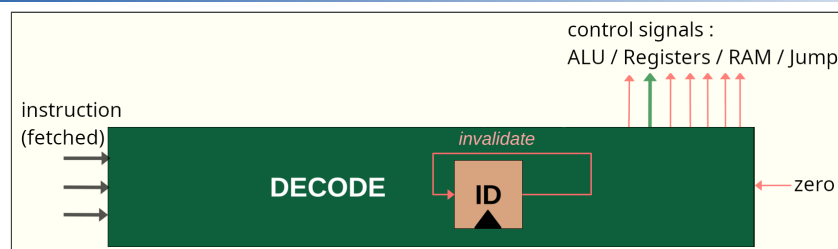


Figure : Decode module

The *decode* block retrieves the *fetch* block output on each clock cycle. Its main role is to check all parts of the instruction are correct (addressing mode, operation value, data source and destination) and transform these values into control signals to be sent to other CPU parts (Addressing Unit, Data path, ...). Actually this block is a mix between a sequential part (storing the correct decoded instruction to be executed on the next clock cycle) and a combinatory part (providing the control signals).

In our CPU, the *decode* block may simply be a register (ID) storing the full 24-bit instruction, such as *fetch* does, and a combinatory part that transform an instruction in control signals.

In this last part, if a jump instruction is detected, and shall be executed on next cycle, the last fetched instruction is no more valid : it is needed to invalidate this last instruction and do not execute it. The ID register input shall be modified according to this state. The other way is to modify the ID register output value and transforms it in a **NOP** (NO Operation, a dummy instruction that does nothing except consuming a clock cycle). This part is a little bit more detailed in the jump instructions (**JNE** and **JEQ**).

There is another behaviour to address : an invalid instruction. A full 24-bit instruction may contain some erroneous data and shall not be executed (at least it will create a bug, at worst it could damage the electronics depending on its design and the generated control signals). That part must be done before storing the instruction in the ID register : that means the *decode* block shall not be totally a sequential circuit with a simple register and an *output clear* signal as described in the previous paragraph : it shall also contain a combinatory circuit in order to detect malformed instructions. This part can be avoided in the context of this course, but is a really important feature to add in a real CPU.

Execute

Execute is not a module as itself, it is more a step in the whole instruction lifecycle. When 2 cycles have already been done, the first instruction is stored in the ID register and is ready to be processed by a full combinatory circuit in order to drive the other CPU parts. The second instruction is stored in the IF register because it comes one cycle later.

The current instruction that will be executed in the next clock rising edge is the instruction stored in ID and is ready to go. The text below illustrates the time shift between 3 following instructions. Each row represents a clock period.

Fetch	Decode	Execute
-none-	-none-	-none-
INSTR-01	-none-	-none-
INSTR-02	INSTR-01	-none-
INSTR-03	INSTR-02	INSTR-01
INSTR-04	INSTR-03	INSTR-02
INSTR-05	INSTR-04	INSTR-03

Addressing Unit

In real CPUs, the instructions can mainly handle several addressing modes. This means the same instruction (for example a subtraction) is able to perform the operation of two operands wherever they are located (register, constant, memory area, ...). This is handled by the Control Unit, providing control signals to the Addressing Unit, in order to redirect values from correct sources to correct destinations.

In order to simplify the whole circuit, in our CPU there is only one addressing mode per instruction. Some will handle constant values or addresses, and some will get values from the register, or the memory.

Code address register

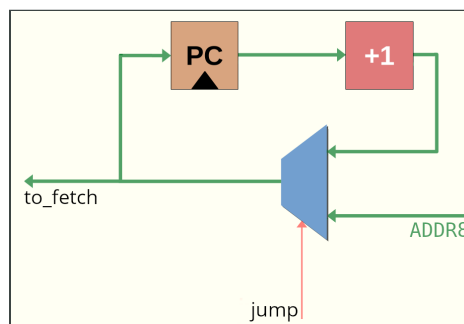


Figure : PC register

The **PC** (Program Counter) register stores the last fetched instruction address. This allows the CPU to know where it is in the program execution. This register only stores an address. That means in our case, the **PC** register is just a 8-bit register with the address of the 24-bit instruction that was fetched when the last clock rising edge occurred.

This register updates its value by increasing the address 1 by 1. In a real CPU, the address could depend on the instruction size, but in our design, the instructions always have the same size, so the address step is always 1.

In case of jump operations, the next address is not the following but another one located at the given address (jump address is usually given in the jump instruction itself). A multiplexer shall be plugged in order to provide the next +1 address or the given jump address. The jump control signal comes from the decode block, itself provided from both the jump instruction code, and the zero flag value (from data path block).

Notice Our design is set to keep in the **PC** the address of the last fetched instruction, so **PC** and **IF** registers are coherent when we pause the simulation. That means the very first address to be fetched will be 0x1 : the instruction @0x0 will never be fetched at start. It could be easily fixed by adding a control signal giving information about CPU startup conditions, but it would add another control signal that is not so relevant for the course purpose. In our design, we do not expect the 0x0 code address to contain something relevant. In order to be sure of that, the CYT-VX8 asm compiler python script always adds a **NOP** operation at the beginning of the code.

Jump instructions

As briefly described in the previous paragraph, the jump instructions (**JNE** and **JEQ**) contain the destination address where the code has to branch to. This address shall be provided to the Addressing Unit in order to be used during the next clock cycle. This allows to retrieve the next instruction at this specific address from the code memory. The Control Unit shall provide the jump signal + the **ADDR8** value in order the jump operation to be effective.

The jump operation is based on the last comparison performed by the ALU (a previous instruction, not necessarily the last executed one). If a comparison was performed, the **RCMP** register would contain a logic value telling if this comparison was ok or not.

In some CPUs, equality comparisons actually perform a subtraction, and if the result is 0, the jump operations can occur. In our CPU design, we keep this convention.

The ALU shall provide the comparison output (called 'zero' here in the previous figures). In case the zero signal is False (meaning the comparison was not ok), and the next jump instruction to be executed is **JNE** (Jump if Not Equal), the jump will occur. If the next jump operation is **JEQ** (Jump if Equal) and the zero signal is True (Meaning the comparison was ok), the jump will occur as well.

Whatever the design implementation, the zero signal (or whatever its name and purpose), it must be compatible with the fact that a **JEQ** instruction will be effective if the last comparison was OK, and a **JNE** instruction will be effective, if the last comparison was not OK.

Another way to explain this behaviour, is to say that **JEQ** instructions are effective when the **RCMP** value is zero, and **JNE** instructions are effective when **RCMP** value is different from zero.

Load and Store instructions

In order to retrieve data from memory, there is only one instruction : **LOAD**. This instruction gets a byte value from the data memory and loads it into one register (from **R1** to **R7**). As previously described, there is no need for the Control Unit to provide a LOAD signal because the data memory output can always be connected to the data bus, and the "load/cnst" multiplexer in front of the data path input will redirect the correct value if needed.

To store data into the memory, the **STOR** instruction shall be executed. the Control Unit must send a store signal to the memory, in order the data memory to be configured correctly and accept data coming from the data path block.

Here again the data memory value shall be previously loaded into a register before calling this instruction, and the data bus comes from the data-path register output and can be directly vconnected to the data memory input bus. That means in our CPU, the **STOR** instruction does not need the Addressing Unit at all to be effective. The involved signals come from the Control Unit and the Data Path, and go to the Data memory.

The destination address to use is coming directly from the instruction itself. The 8-bit address (**ADDR8**) is directly sent to the memory address bus.

There is a huge limitation, due to the poor variety of addressing modes. Any constant that needs to be stored in memory must be loaded into a register before. It also consumes an additional cycle in order to perform this kind of operation.

Addressing registers

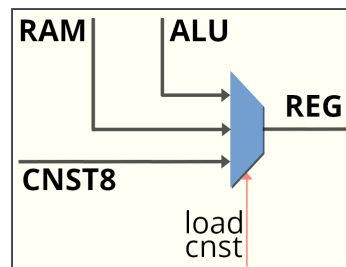


Figure : Addressing registers

When an instruction needs to provide a value into any register (from **R1** to **R7**), the Addressing Unit has to branch the data source to the data-path block correctly. In order to do this, the Control Unit sends some control signals such as **CNST** instruction, or **LOAD** instruction or any other information about the current operation to be performed. Then the Addressing Unit, via a multiplexer, selects the correct source between data RAM memory, 8-bit constant coming from the instruction directly, or the ALU output that needs to be stored in the destination register.

Using constants

In the CYT-VX8 design, the addressing modes are really simplified and it is not possible to perform operations without registers (with constants for example). That means it is not possible to directly perform an operation between a register and a constant.

In order to use a constant, it shall be loaded into a register first. This design simplifies the circuit needed for the ALU and the whole data path, but it makes the program slower because everytime a constant needs to be used, an additional clock cycle is consumed.

The **CNST** instruction takes a 8-bit value and the destination register number (from **R1** to **R7**).

As shown in the previous figure, the constant values shall be loaded in a register, that means the register input cannot be the ALU output directly. Here it is needed to retrieve the 8-bit constant value (**CNST8**) value from the instruction itself : this is a task performed by the Control Unit.

The Addressing Unit shall implement a multiplexer in order to choose between ALU output and constant value.

Memory interface

In the simplified design of CYT-VX8, the memory access is splitted in several paths in order to distinguish each access mode easily.

As already said, our architecture is based on the Harvard's one, that means the code and the data are not located in the same memory. This split allows to retrieve the next code instruction while performing a memory operation at the same time.

This is not the only design choice about memory access : the data memory shall be accessed in read mode and write mode separately. That means the data bus shall be splitted into two : one coming from the data path block, the other going back to it. The data memory input will be any register output, and the data memory output will be stored into any data path register.

Description

This block is a sequential one, synchronized on the clock cycle. Whatever the memory technology, it shall provide a read and write access to data memory, and a read only access to code memory.

Each data address contains a 8-bit value, and the address bus are 8-bit lengths too. This is a huge limitation, clamping the total data memory size to 256 bytes. For the education purpose this will be widely sufficient, but in a real CPU the address bus should be wider.

Interface diagram

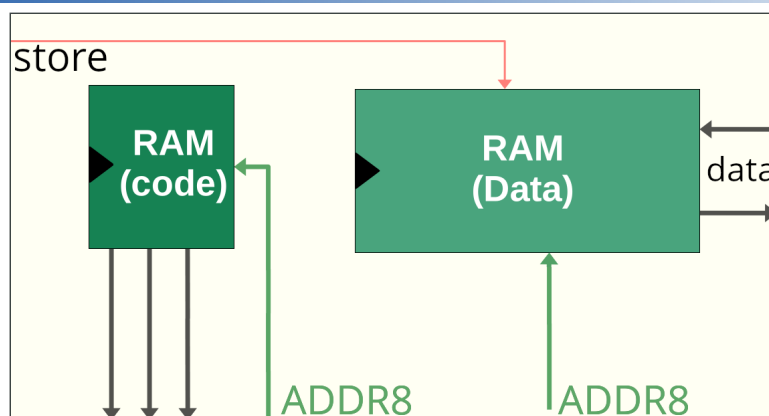


Figure : Memory interface

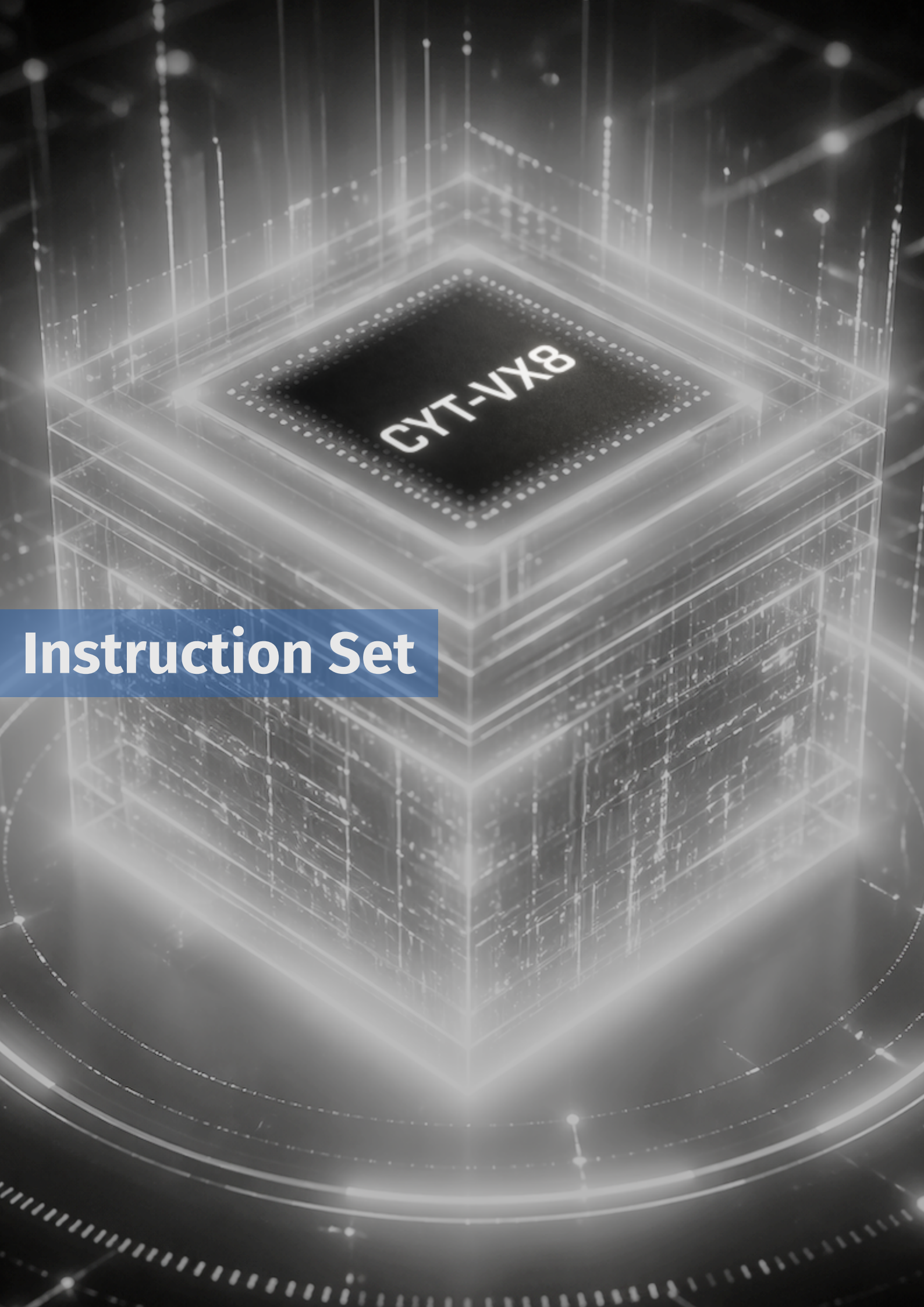
In the figure there is a store signal coming from the control unit in order to tell if the data input shall be stored at the given address or not. The load signal could be there too, but in our design, as the data memory has two buses, it is not an issue to always have the data being sent to the output.

There are two address signals, one for the code (to retrieve the instructions) and one for the data (to be read or written).

The data input comes from the register output. This is the same signal than the first ALU input. that means we can directly link a register output to the data memory.

The data output will be sent to the addressing unit, in order to be redirected to the register input if a STOR instruction is being executed, else this output will not be used at all.

The code memory output will contain



CYT-VX8

Instruction Set

Instruction set

Description

In the following pages, are described all the instruction details for the CYT-VX8 CPU.

For any instruction you can find :

- the MNEMONIC (name)
- the human-readable assembly format
- the 24-bits binary format
- the hexadecimal format
- a functional+technical description

This list is not sorted in alphabetical order, but instructions are gathered by functional types.

- arithmetic operations (2 registers)
- arithmetic operations (1 register)
- logic operations (2 registers)
- logic operations (1 register)
- value comparisons
- jump operations
- data memory access
- other specific instructions

If you need to have a sorted list of instructions, please refer to the appendix at the end of this document.

Arithmetic - 2 registers

ADD

Assembly instruction

```
ADD, #Rin1, #Rin2, #Rout
```

Binary / Hex format

```
0001 1000 #Rin1 #Rin2 0000 #Rout 18 X X 0 X
```

1 cycle

This instruction performs the addition of 2 registers and stores the result in a third one. The operation will perform a classic binary sum.

#Rin1 is the input register number (4bits) from R1 to R7.
 #Rin2 is the input register number (4bits) from R1 to R7.
 #Rout is the output register number (4bits) from R1 to R7.

SUB

Assembly instruction

```
SUB, #Rin1, #Rin2, #Rout
```

Binary / Hex format

```
0001 1001 #Rin1 #Rin2 0000 #Rout 19 X X 0 X
```

1 cycle

This instruction performs the subtraction of 2 registers and stores the result in a third one. The #Rin2 register value is subtracted from the #Rin1 register value. The output is stored in the #Rout register.

#Rin1 is the input register number (4bits) from R1 to R7.
 #Rin2 is the input register number (4bits) from R1 to R7.
 #Rout is the output register number (4bits) from R1 to R7.

Notice The register value representation is assumed to be two's complement : all values are expected to be signed integers.

MAX

Assembly instruction

```
MAX, #Rin1, #Rin2, #Rout
```

Binary / Hex format

```
0001 1011 #Rin1 #Rin2 0000 #Rout 1B X X 0 X
```

1 cycle

This instruction compares 2 register values and store the biggest value into the output register.

#Rin1 is the input register number (4bits) from R1 to R7.

#Rin2 is the input register number (4bits) from R1 to R7.

#Rout is the output register number (4bits) from R1 to R7.

Notice The register value representation is assumed to be classic binary : all values are expected to be natural integers.

MIN

Assembly instruction

```
MIN, #Rin1, #Rin2, #Rout
```

Binary / Hex format

```
0001 1010 #Rin1 #Rin2 0000 #Rout 1A X X 0 X
```

1 cycle

This instruction compares 2 register values and store the lowest value into the output register.

#Rin1 is the input register number (4bits) from R1 to R7.

#Rin2 is the input register number (4bits) from R1 to R7.

#Rout is the output register number (4bits) from R1 to R7.

Notice The register value representation is assumed to be classic binary : all values are expected to be natural integers.

Arithmetic - 1 register

INC

Assembly instruction

INC, #Rin1

Binary / Hex format

0001 1100 #Rin1 0000 0000 #Rin1 1C X 0 0 X

1 cycle

This instruction increases the value of a register by 1. The ALU performs this operation from its #1 input. No ABS operation can be performed from #2 input. The ALU output is expected to be the same register than input.

The assembly instruction takes only one input register reference. The compiler will provide the same value for the #Rout reference in order to update the current selected register. There is no technical limitations to the output register value : it could be possible to force the result of the absolute value to another register. This is an arbitrary compiler limitation. The reason is to simplify the whole circuit : the output register is always given in the least significant nibble.

#Rin1 is the input/output register number (4bits) from R1 to R7.

DEC

Assembly instruction

DEC, #Rin1

Binary / Hex format

0001 1101 #Rin1 0000 0000 #Rin1 1D X 0 0 X

1 cycle

This instruction decreases the value of a register by 1. The ALU performs this operation from its #1 input. No ABS operation can be performed from #2 input. The ALU output is expected to be the same register than input.

The assembly instruction takes only one input register reference. The compiler will provide the same value for the #Rout reference in order to update the current selected register. There is no technical limitations to the output register value : it could be possible to force the result of the absolute value to another register. This is an arbitrary compiler limitation. The reason is to simplify the whole circuit : the output register is always given in the least significant nibble.

#Rin1 is the input/output register number (4bits) from R1 to R7.

ABS

Assembly instruction

```
ABS, #Rin1
```

Binary / Hex format

```
0001 1111 #Rin1 0000 0000 #Rin1 1F X 0 0 X
```

1 cycle

This instruction computes the absolute value of a register. The register value representation is assumed to be *two's complement*. The ALU performs this operation from its #1 input. No ABS operation can be performed from #2 input. The ALU output is expected to be the same register than input.

The assembly instruction takes only one input register reference. The compiler will provide the same value for the `#Rout` reference in order to update the current selected register. There is no technical limitations to the output register value : it could be possible to force the result of the absolute value to another register. This is an arbitrary compiler limitation. The reason is to simplify the whole circuit : the output register is always given in the least significant nibble.

`#Rin1` is the input/output register number (4bits) from `R1` to `R7`.

NEG

Assembly instruction

```
NEG, #Rin1
```

Binary / Hex format

```
0001 1110 #Rin1 0000 0000 #Rin1 1E X 0 0 X
```

1 cycle

This instruction toggles the sign bit of a register value. The register value representation is assumed to be *two's complement*. The ALU performs this operation from its #1 input. No NEG operation can be performed from #2 input. The ALU output is expected to be the same register than input.

The assembly instruction takes only one input register reference. The compiler will provide the same value for the `#Rout` reference in order to update the current selected register. There is no technical limitations to the output register value : it could be possible to force the result of the absolute value to another register. This is an arbitrary compiler limitation. The reason is to simplify the whole circuit : the output register is always given in the least significant nibble.

`#Rin1` is the input register number (4bits) from `R1` to `R7`.

Logic - 2 registers

AND

Assembly instruction

```
AND, #Rin1, #Rin2, #Rout
```

Binary / Hex format

```
0001 0000 #Rin1 #Rin2 0000 #Rout 10 X X 0 X
```

1 cycle

This instruction performs a bitwise-AND operation between 2 register values and stores the result into the output register.

#Rin1 is the input register number (4bits) from R1 to R7.
 #Rin2 is the input register number (4bits) from R1 to R7.
 #Rout is the output register number (4bits) from R1 to R7.

Notice The register value representation is not used here. All values are expected to be natural integers.

OR

Assembly instruction

```
OR, #Rin1, #Rin2, #Rout
```

Binary / Hex format

```
0001 0001 #Rin1 #Rin2 0000 #Rout 11 X X 0 X
```

1 cycle

This instruction performs a bitwise-OR operation between 2 register values and stores the result into the output register.

#Rin1 is the input register number (4bits) from R1 to R7.
 #Rin2 is the input register number (4bits) from R1 to R7.
 #Rout is the output register number (4bits) from R1 to R7.

Notice The register value representation is not used here. All values are expected to be natural integers.

XOR

Assembly instruction

```
XOR, #Rin1, #Rin2, #Rout
```

Binary / Hex format

```
0001 0010 #Rin1 #Rin2 0000 #Rout 12 X X 0 X
```

1 cycle

This instruction performs a bitwise-Exclusive-OR operation between 2 register values and stores the result into the output register.0

#Rin1 is the input register number (4bits) from R1 to R7 .
 #Rin2 is the input register number (4bits) from R1 to R7 .
 #Rout is the output register number (4bits) from R1 to R7 .

Notice The register value representation is not used here. All values are expected to be natural integers.

NAND

Assembly instruction

```
NAND, #Rin1, #Rin2, #Rout
```

Binary / Hex format

```
0001 0011 #Rin1 #Rin2 0000 #Rout 13 X X 0 X
```

1 cycle

This instruction performs a bitwise-Not-AND operation between 2 register values and stores the result into the output register.

#Rin1 is the input register number (4bits) from R1 to R7 .
 #Rin2 is the input register number (4bits) from R1 to R7 .
 #Rout is the output register number (4bits) from R1 to R7 .

Notice The register value representation is not used here. All values are expected to be natural integers.

NOR

Assembly instruction

```
NOR, #Rin1, #Rin2, #Rout
```

Binary / Hex format

```
0001 0100 #Rin1 #Rin2 0000 #Rout 14 X X 0 X
```

1 cycle

This instruction performs a bitwise-Not-OR operation between 2 register values and stores the result into the output register.

#Rin1 is the input register number (4bits) from R1 to R7 .
 #Rin2 is the input register number (4bits) from R1 to R7 .
 #Rout is the output register number (4bits) from R1 to R7 .

Notice The register value representation is not used here. All values are expected to be natural integers.

XNOR

Assembly instruction

```
NXOR, #Rin1, #Rin2, #Rout
```

Binary / Hex format

0001 0101	#Rin1	#Rin2	0000	#Rout	15	X	X	0	X
-----------	-------	-------	------	-------	----	---	---	---	---

1 cycle

This instruction performs a bitwise-Not-Exclusive-OR operation between 2 register values and stores the result into the output register.

#Rin1 is the input register number (4bits) from R1 to R7 .
#Rin2 is the input register number (4bits) from R1 to R7 .
#Rout is the output register number (4bits) from R1 to R7 .

Notice The register value representation is not used here. All values are expected to be natural integers.

Logic - 1 register

NOT

Assembly instruction

```
NOT, #Rin1, #Rout
```

Binary / Hex format

```
0000 1000 #Rin1 0000 0000 #Rout 08 X 0 0 X
```

1 cycle

This instruction toggles each bit of a register value. The ALU performs this operation from its #1 input. No **NOT** operation can be performed from #2 input. The ALU output can be any register.

The assembly instruction takes two register references : 1 input and 1 output. Here the assembly syntax is not similar to the Logic-1-register instructions : the reason is to create various types of instructions and let students see how to handle these differences. Indeed, the output register number is still stored in the least significant nibble.

#Rin1 is the input register number (4bits) from R1 to R7.
#Rout is the output register number (4bits) from R1 to R7.

LSL

Assembly instruction

```
LSL, #Rin1, #Rout
```

Binary / Hex format

```
0000 1001 #Rin1 0000 0000 #Rout 09 X 0 0 X
```

1 cycle

This instruction performs a 1-bit logical shift to the left. The incoming bits will be set to 0. The ALU performs this operation from its #1 input. No **LSL** operation can be performed from #2 input. The ALU output can be any register.

The assembly instruction takes two register references : 1 input and 1 output. Here the assembly syntax is not similar to the Logic-1-register instructions : the reason is to create various types of instructions and let students see how to handle these differences. Indeed, the output register number is still stored in the least significant nibble.

#Rin1 is the input register number (4bits) from R1 to R7.
#Rout is the output register number (4bits) from R1 to R7.

Notice : If the value is a signed one, the sign bit (bit7) may be changed, depending on bit6 value.

LSR

Assembly instruction

```
LSR, #Rin1, #Rout
```

Binary / Hex format

```
0000 1010 #Rin1 0000 0000 #Rout 0A X 0 0 X
```

1 cycle

This instruction performs a 1-bit logical shift to the right. The most significant bit is filled with a 0 value. The ALU performs this operation from its #1 input. No **LSR** operation can be performed from #2 input. The ALU output can be any register.

The assembly instruction takes two register references : 1 input and 1 output. Here the assembly syntax is not similar to the Logic-1-register instructions : the reason is to create various types of instructions and let students see how to handle these differences. Indeed, the output register number is still stored in the least significant nibble.

#Rin1 is the input register number (4bits) from R1 to R7.
#Rout is the output register number (4bits) from R1 to R7.

Notice : If the value is a signed one, the sign bit will be set to positive (and may be removed in case of negative value). If you want to keep the sign bit, check ASR instruction.

ASR

Assembly instruction

```
ASR, #Rin1, #Rout
```

Binary / Hex format

```
0000 1011 #Rin1 0000 0000 #Rout 0B X 0 0 X
```

1 cycle

This instruction performs a 1-bit arithmetic shift to the right. The register value representation is assumed to be *two's complement*. The most significant bit (left) will be duplicated during the shift. The ALU performs this operation from its #1 input. No **ASR** operation can be performed from #2 input. The ALU output can be any register.

The assembly instruction takes two register references : 1 input and 1 output. Here the assembly syntax is not similar to the Logic-1-register instructions : the reason is to create various types of instructions and let students see how to handle these differences. Indeed, the output register number is still stored in the least significant nibble.

#Rin1 is the input register number (4bits) from R1 to R7.
#Rout is the output register number (4bits) from R1 to R7.

ROL

Assembly instruction

```
ROL, #Rin1, #Rout
```

Binary / Hex format

```
0000 1100 #Rin1 0000 0000 #Rout 0C X 0 0 X
```

1 cycle

This instruction performs a 1-bit rotary shift to the left. The most significant bit will become the less significant one after rotation. All other bits are simply shifted to the left. The ALU performs this operation from its #1 input. No **ROL** operation can be performed from #2 input. The ALU output can be any register.

The assembly instruction takes two register references : 1 input and 1 output. Here the assembly syntax is not similar to the Logic-1-register instructions : the reason is to create various types of instructions and let students see how to handle these differences. Indeed, the output register number is still stored in the least significant nibble.

#Rin1 is the input register number (4bits) from R1 to R7.
#Rout is the output register number (4bits) from R1 to R7.

Notice : If the value is a signed one, the sign bit (bit7) may be changed, depending on bit6 value.

ROR

Assembly instruction

```
ROR, #Rin1, #Rout
```

Binary / Hex format

```
0000 1101 #Rin1 0000 0000 #Rout 0D X 0 0 X
```

1 cycle

This instruction performs a 1-bit rotary shift to the right. The less significant bit will become the most significant one after rotation. All other bits are simply shifted to the right. The ALU performs this operation from its #1 input. No **ROR** operation can be performed from #2 input. The ALU output can be any register.

The assembly instruction takes two register references : 1 input and 1 output. Here the assembly syntax is not similar to the Logic-1-register instructions : the reason is to create various types of instructions and let students see how to handle these differences. Indeed, the output register number is still stored in the least significant nibble.

#Rin1 is the input register number (4bits) from R1 to R7.
#Rout is the output register number (4bits) from R1 to R7.

Notice : If the value is a signed one, the sign bit (bit7) may be changed, depending on bit0 value.

Data comparisons

CMP_LT

Assembly instruction

```
CMP_LT, #Rin1, #Rin2
```

Binary format | Hex format

```
0000 0001 #Rin1 #Rin2 0000 0111 01 X X 07
```

1 cycle

This instruction compares both ALU input values, coming from any registers. The result will be stored in the RCMP register (technically this could be stored in any register but here we force the ALU output to be redirected to the RCMP : this is just an arbitrary choice). This instruction will check if #Rin1 register value is Lesser Than #Rin2 register value.

#Rin1 is the input register number (4bits) from R1 to R7 .

#Rin2 is the input register number (4bits) from R1 to R7 .

CMP_LE

Assembly instruction

```
CMP_LE, #Rin1, #Rin2
```

Binary / Hex format

```
0000 0010 #Rin1 #Rin2 0000 0111 02 X X 07
```

1 cycle

This instruction compares both ALU input values, coming from any registers. The result will be stored in the RCMP register (technically this could be stored in any register but here we force the ALU output to be redirected to the RCMP : this is just an arbitrary choice). This instruction will check if #Rin1 register value is Lesser or Equal to #Rin2 register value.

#Rin1 is the input register number (4bits) from R1 to R7 .

#Rin2 is the input register number (4bits) from R1 to R7 .

CMP_EQ

Assembly instruction

```
CMP_EQ, #Rin1, #Rin2
```

Binary / Hex format

```
0000 0011 #Rin1 #Rin2 0000 0111 03 X X 07
```

1 cycle

This instruction compares both ALU input values, coming from any registers. The result will be stored in the **RCMP** register (technically this could be stored in any register but here we force the ALU output to be redirected to the **RCMP** : this is just an arbitrary choice). This instruction will check if **#Rin1** register value is Equal to **#Rin2** register value.

#Rin1 is the input register number (4bits) from **R1** to **R7** .
#Rin2 is the input register number (4bits) from **R1** to **R7** .

CMP_GE

Assembly instruction

```
CMP_GE, #Rin1, #Rin2
```

Binary / Hex format

```
0000 0100 #Rin1 #Rin2 0000 0111 04 X X 07
```

1 cycle

This instruction compares both ALU input values, coming from any registers. The result will be stored in the **RCMP** register (technically this could be stored in any register but here we force the ALU output to be redirected to the **RCMP** : this is just an arbitrary choice). This instruction will check if **#Rin1** register value is Greater or Equal to **#Rin2** register value.

#Rin1 is the input register number (4bits) from **R1** to **R7** .
#Rin2 is the input register number (4bits) from **R1** to **R7** .

CMP_GT

Assembly instruction

```
CMP_GT, #Rin1, #Rin2
```

Binary / Hex format

```
0000 0101 #Rin1 #Rin2 0000 0111 05 X X 07
```

1 cycle

This instruction compares both ALU input values, coming from any registers. The result will be stored in the **RCMP** register (technically this could be stored in any register but here we force the ALU output to be redirected to the **RCMP** : this is just an arbitrary choice). This instruction will check if **#Rin1** register value is Greater Than **#Rin2** register value.

#Rin1 is the input register number (4bits) from **R1** to **R7** .
#Rin2 is the input register number (4bits) from **R1** to **R7** .

CMP_NE

Assembly instruction

```
CMP_NE, #Rin1, #Rin2
```

Binary / Hex format

```
0000 0110 #Rin1 #Rin2 0000 0111 06 X X 07
```

1 cycle

This instruction compares both ALU input values, coming from any registers. The result will be stored in the RCMP register (technically this could be stored in any register but here we force the ALU output to be redirected to the RCMP : this is just an arbitrary choice). This instruction will check if #Rin1 register value is Not Equal to #Rin2 register value.

#Rin1 is the input register number (4bits) from R1 to R7 .
#Rin2 is the input register number (4bits) from R1 to R7 .

Jump instructions

The Control Unit may generate a jump signal on either **JMP**, **JNE** or **JEQ** instructions.

The jump signal is set when a **JMP** instruction occurs, without any specific condition.

When either a **JNE** or **JEQ** instruction occurs, the jump signal is not a direct bitwise-OR-operation between these two states : in both cases, the content of **RCMP** is used in addition. Jump signal is generated if either there is a **JEQ** instruction + **RCMP** equals to 0, or if there is a **JNE** instruction and the **RCMP** content is not equal to 0.

In all cases, if the jump signal is enabled (if the jump has to be executed properly), the previous fetched instruction is not correct anymore : in this case, the Control Unit invalidates the next instruction to be decoded (the one coming from the fetch stage). Invalidating means the last fetched instruction shall be transformed into a **NOP** operation on the next clock rising edge.

Invalidating an instruction, will make the CPU lose a cycle (**NOP** operation is executed) : that means any jump instruction takes 2 cycles on this CPU.

If the jump signal is disabled, the CPU continues its classic behaviour (fetch, decode, execute).

The jump signal is sent to the Addressing Unit, in order to select which address value to send to the RAM (code) : either the **PC** + 1, or the instruction nested address value.

JMP

Assembly instruction

JMP, ADDR8

Binary / Hex format

0010 0101 0000 **ADDR8** 0000 25 0 **XX** 0

2 cycles

Jump with no condition. This instruction is always executed.

JEQ

Assembly instruction

JEQ, ADDR8

Binary / Hex format

0010 0111 0000 **ADDR8** 0000 27 0 **XX** 0

2 cycles

Jump if Equal. This instruction is executed if the last comparison was correct.

JNE

Assembly instruction

JNE, ADDR8

Binary / Hex format

0010 0110 0000 ADDR8 0000

26 0 XX 0

2 cycles

Jump if Not Equal. This instruction is executed if the last comparison was **not** correct.

Memory access

LOAD

Assembly instruction

```
LOAD, ADDR8, #Rout
```

Binary / Hex format

```
0010 0001 0000 ADDR8 #Rout 21 0 XX X
```

1 cycle

This instruction reads a byte from the Data memory and stores this value into a register.

The ADDR8 8-bit address is given as the absolute source address. If the PAGE instruction has been implemented, this address is an offset relative to the current page.

The #Rout value is the destination register number (from R1 to R7).

STOR

Assembly instruction

```
STOR, #Rin1, ADDR8
```

Binary / Hex format

```
0010 0010 #Rin1 ADDR8 0000 22 #Rin1 XX 0
```

1 cycle

This instruction reads a register value and loads it into the data memory..

The ADDR8 8-bit address is given as the absolute source address. If the PAGE instruction has been implemented, this address is an offset relative to the current page.

The #Rin1 value is the destination register number (from R1 to R7). STOR instructions will only take register values from the first ALU input.

PAGE

Assembly instruction

```
PAGE, CNST8, CNST8
```

Binary / Hex format

```
0010 0011 CNST8 CNST8 23 XX XX
```

1 cycle

To be defined for a future use. This instruction is intended to be used to configure a memory page number, so then the classic STOR and LOAD instructions will be able to read/write data but in a larger scale.

Others

CNST

Assembly instruction

```
CNST, CNST8, #Rout
```

Binary / Hex format

```
0010 0100 0000 CNST8 #Rout 24 0 XX X
```

1 cycle

This instruction will store an 8-bit constant value (provided in the instruction itself) and stores it into a register.

The **#Rout** value is the destination register number (from **R1** to **R7**).

This instruction is there only to simplify the CPU circuit (especially the DECODE part of the Control Unit), but in a real CPU this instruction would be totally useless. That said, this instruction makes the whole program lose some time (1 cycle) when the program needs to handle constant values.

NOP

Assembly instruction

```
NOP
```

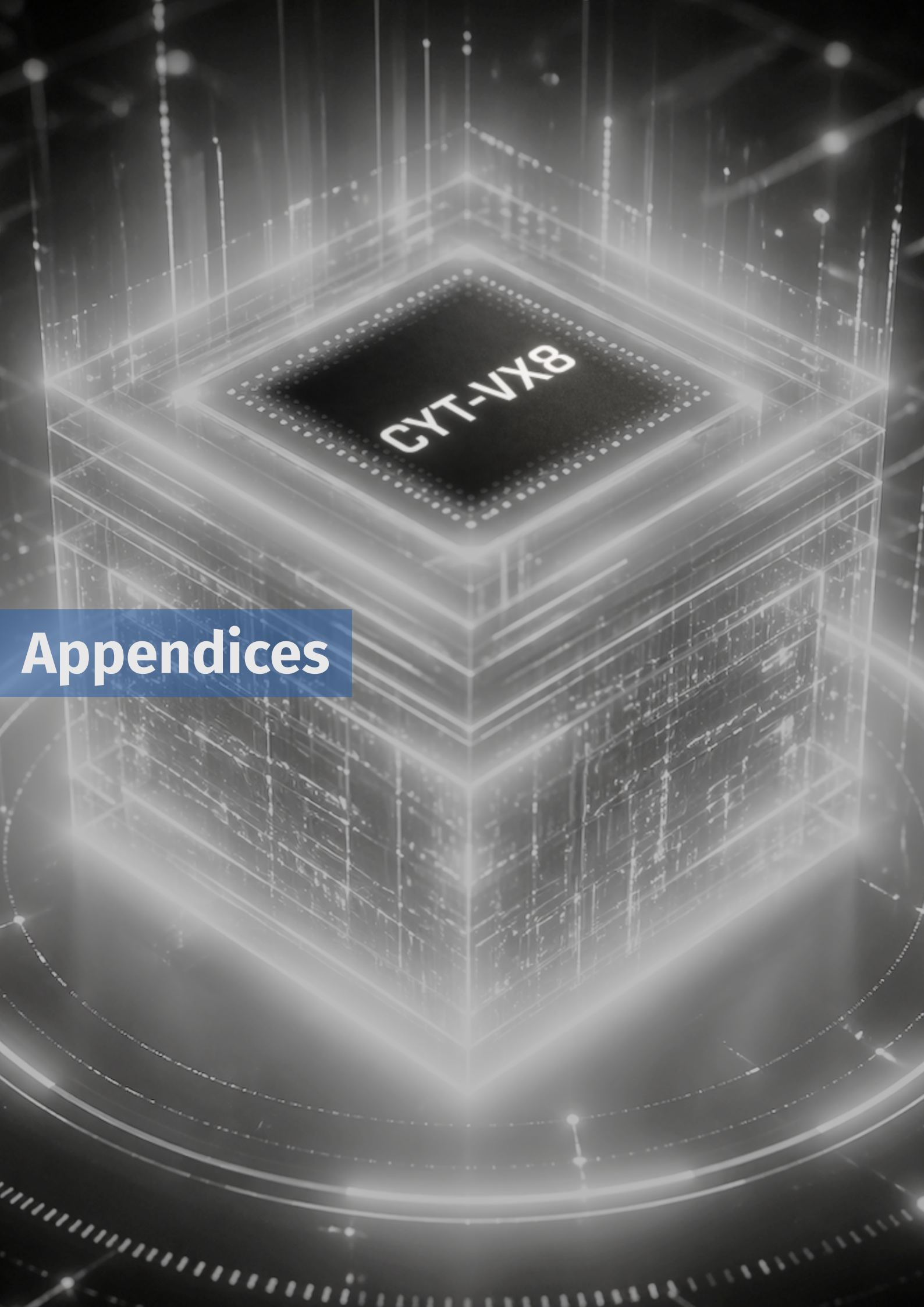
Binary / Hex format

```
0000 0000 0000 0000 0000 0000 00 00 00
```

1 cycle

This instruction is a dummy instruction that only makes the CPU consume 1-cycle. The 24-bit are all set to zero. This instruction is not intended to produce any specific behaviour : that means, all data registers and all data memory bytes must remain unchanged. The ALU is not expected to perform any operation (if it does, the result won't be stored anywhere).

This instruction is important to be able to simplify the JUMP operations, as the previous fetched instruction will have to be invalidated if a branch occurs : in this case, the simplification is to transform this fetched instruction into a NOP instruction. The jump will occur, and the next instruction to be executed will be this new NOP. It silently shadows the branch behaviour.



CYT-VX8

Appendices

Index

Instruction index

◆ABS	30
◆ADD	27
◆AND	31
◆ASR	35
◆CMP_EQ	38
◆CMP_GE	38
◆CMP_GT	38
◆CMP_LT	37
◆CMP_LE	37
◆CMP_NE	39
◆CNST	43
◆DEC	29
◆INC	29
◆JEQ	40
◆JMP	40
◆JNE	41
◆LOAD	42
◆LSL	34
◆LSR	35
◆MAX	28
◆MIN	28
◆NAND	32
◆NEG	30
◆NOR	32
◆NOT	34
◆OR	31

◆PAGE	42
◆ROL	36
◆ROR	36
◆STOR	42
◆SUB	27
◆XNOR	33
◆XOR	32

Figure Index

◆Addressing registers	21
◆ALU	14
◆Data-Path	15
◆Decode (unit control)	17
◆Fetch (unit control)	17
◆Main diagram	10
◆Memory interface	22
◆PC (Program Counter)	19
◆Registers	15

Assembly code examples

Add & Subtract

C language	CYT-VX8 asm	Address Bytecode
// &u1 => 0x00, &u2 => 0x01	NOP	00 000000
unsigned char u1 = 43;	CNST, 2B, R1	01 2402B0
unsigned char u2 = 89;	CNST, 59, R2	02 240591
// &s1 => 0x02, &s2 => 0x03	CNST, DB, R3	03 240DB2
unsigned char s1 = -37;	CNST, 73, R4	04 240733
unsigned char s2 = 115;	STOR, R1, 00	05 220000
	STOR, R2, 01	06 221010
	STOR, R3, 02	07 222020
	STOR, R4, 03	08 223030
 // ADD :		
// &res1 => 0x04, &res2 => 0x05	ADD, R1, R2, R5	09 180104
unsigned char res1 = u1 + u2;	ADD, R3, R4, R6	0A 182305
char res2 = s1 + s2;	STOR, R5, 04	0B 224040
	STOR, R6, 05	0C 225050
 // SUB :		
// &res3 => 0x06, &res2 => 0x07	SUB, R1, R2, R5	0D 190104
unsigned char res3 = u1 - u2;	SUB, R3, R4, R6	0E 192305
char res4 = s1 - s2;	STOR, R5, 06	0F 224060
	STOR, R6, 07	10 225070

Change sign & Maximum value

C language	CYT-VX8 asm	Address Bytecode
// &a => 0xAA	NOP	00 000000
// &b => 0xBB		
// &c => 0xCC		
char a = 57;	CNST, 39, R1	01 240390
char b = -98;	CNST, 9E, R2	02 2409E1
char c;	STOR, R1, AA	03 220AA0
	STOR, R2, BB	04 221BB0
 a = -a;	NEG, R1	05 1E0000
c = a > b ? a : b;	MAX, R1, R2, R3	06 1B0102
	STOR, R3, CC	07 222CC0

Bitwise Operations

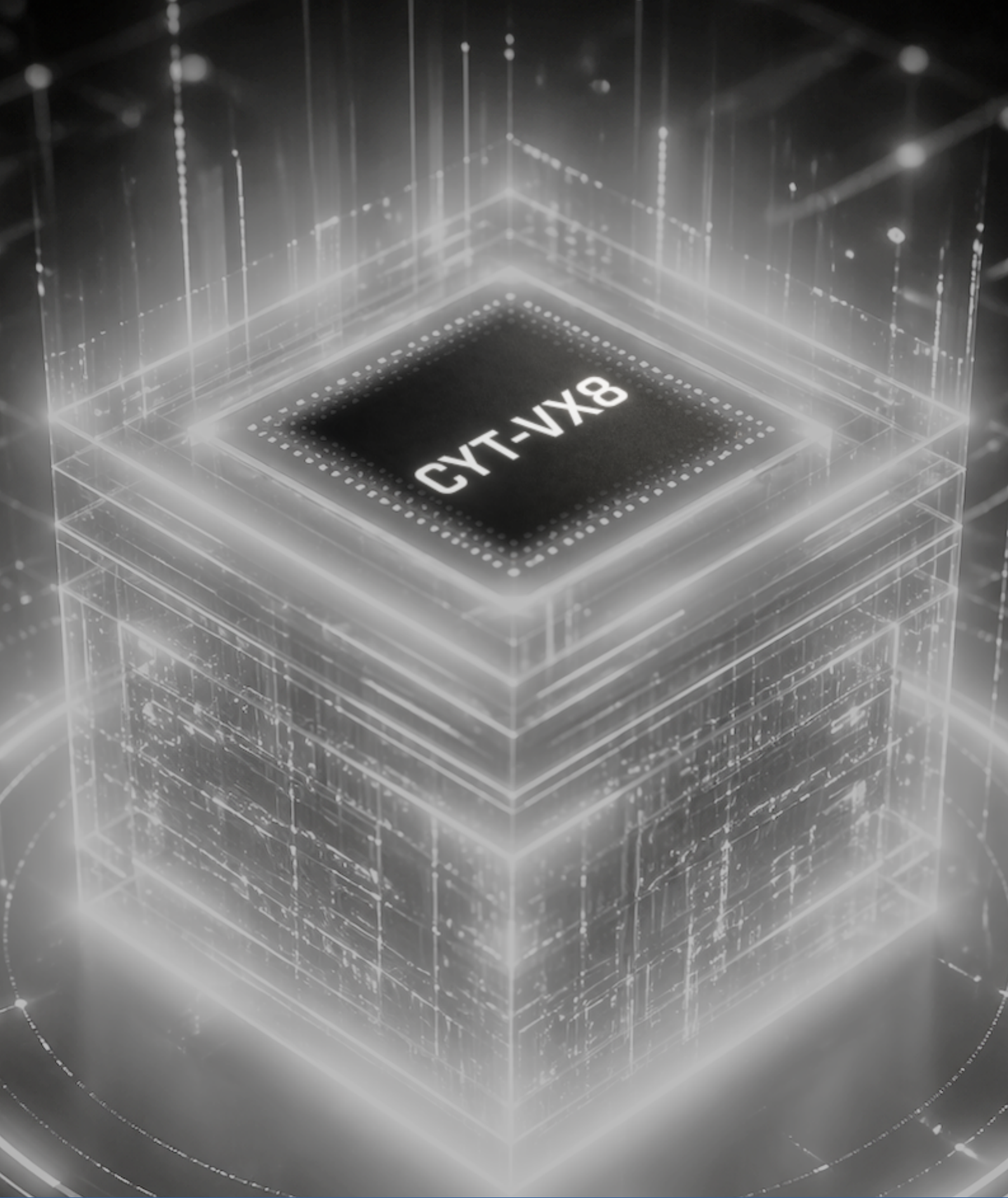
C language	CYT-VX8 asm	Address Bytecode
// &u0, &u6 => 0x00, 0x06	NOP	00 000000
char u0 = 0xA5;	CNST, A5, R1	01 240A50
char u1, u2, u3, u4, u5, u6;	STOR, R1, 00	02 220000
// And		
u1 = u0 & 0xF0;	CNST, F0, R7	03 240F06
	AND, R1, R7, R2	04 100601
	STOR, R2, 01	05 221010
// Or		
u2 = u0 0xC3;	CNST, C3, R7	06 240C36
	OR, R1, R7, R2	07 110601
	STOR, R2, 02	08 221020
// Xor		
u3 = u0 ^ 0x3C;	CNST, 3C, R7	09 2403C6
	XOR, R1, R7, R2	0A 120601
	STOR, R2, 03	0B 221030
// Complement		
u4 = ~u0;	NOT, R1, R2	0C 080001
	STOR, R2, 04	0D 221040
// Left Shift		
u5 = u0 << 1;	LSL, R1, R2	0E 090001
	STOR, R2, 05	0F 221050
// Right shift (arithm.)		
u6 = u0 >> 1;	ASR, R1, R2	10 0B0001
	STOR, R2, 06	11 221060

Sum of integers

C language	CYT-VX8 asm	Address Bytecode
// i is used in a register only	NOP	00 000000
// &k => 0x00	# i, const and k	
unsigned char k = 0;	CNST, 01, R1	01 240010
	CNST, 0F, R2	02 2400F1
	CNST, 00, R3	03 240002
	# Loop start (check)	
for(unsigned char i=1;i<15;i++){	CMP_LE, R1, R2	04 020107
	JNE, 09	05 260090
k += i;	ADD, R3, R1, R3	06 182002
	# Loop end (incr + back)	
	INC, R1	07 1C0000
	JMP, 04	08 250040
}	# store k	
	STOR, R3, 00	09 222000

Switch case function

C language	CYT-VX8 asm	Address Bytecode
// &a => 0xAD char a = ???;	NOP	00 000000
void change_value(){ switch(a){	LOAD AD, R1	01 210AD0
	CNST, 03, R7	02 240036
	CMP_EQ, R1, R7	03 030607
	JEQ, 0C	04 2700C0
	CNST, 05, R7	05 240056
	CMP_EQ, R1, R7	06 030607
	JEQ, 0F	07 2700F0
	CNST, 07, R7	08 240076
	CMP_EQ, R1, R7	09 030607
	JEQ, 12	0A 270120
	JMP, 14	0B 250140
case 3:		
a = a - 11;	CNST, 0B, R7	0C 2400B6
break;	SUB, R1, R7, R1	0D 190600
case 5;	JMP, 15	0E 250150
case 5;	CNST, 0D, R7	0F 2400D6
a = a + 13;	ADD, R1, R7, R1	10 180600
break;	JMP, 15	11 250150
case 7;		
a = a << 1;	LSL, R1, R1	12 090000
break;	JMP, 15	13 250150
default:		
a = 0;	CNST, 00, R1	14 240000
break;		
}		
}	STOR, R1, AD	15 220AD0



Computer Science Department
CY-Tech Engineer School
CY Cergy/Paris University



Avenue du Parc
95000 Cergy-Pontoise Cedex
FRANCE



Eva ANSERMIN
Romuald GRIGNON

eva.ansermin@cyu.fr
romuald.grignon@cyu.fr